

Java 后端项目技术栈重点复习清单

面向“校园资料共享与智能检索平台”的实习面试复习版

复习目标：能结合项目讲清楚每个技术栈在哪里用、为什么用、怎么实现、有哪些坑、如何优化。

复习维度	你需要达到的效果
项目使用场景	能说出该技术在项目哪个模块使用，例如登录、缓存、搜索、限流、下载统计。
核心原理	能解释关键概念，例如 Spring 自动配置、Redis ZSet、MySQL 索引和事务。
关键用法	能写出或看懂常用注解、配置、Mapper、RedisTemplate API。
面试追问	能回答“为什么这样设计”“不用会怎样”“异常情况怎么处理”。
项目化表达	把八股答案改成和自己项目强相关的回答。

建议复习顺序：Spring Boot / SSM -> MyBatis -> MySQL -> Redis -> JWT 权限 -> 文件上传 -> 定时任务 / MQ -> Maven / Git -> 项目综合追问。

一、项目技术栈总览

这个项目不要按“背八股”的方式复习，而应按项目请求链路复习：Controller 接收请求，Service 处理业务，Mapper 操作 MySQL，Redis 承担缓存、排行榜、限流和统计，JWT / 拦截器负责认证，文件上传和审核状态流转体现非 CRUD 业务价值。

项目模块	主要技术栈	面试重点
登录注册	Spring Boot、Spring MVC、JWT、拦截器、MySQL	Token 生成与校验、密码加密、角色权限、登录态过期。
资料上传	Spring MVC MultipartFile、MySQL、MD5	文件校验、文件去重、文件表与资料表拆分。
资料审核	MySQL 事务、状态流转、权限控制	待审核/通过/拒绝/下架，非法状态转换防护。
资料搜索	MyBatis 动态 SQL、MySQL 索引，可扩展 ES	条件查询、status 固定过滤、索引优化。
资料详情	Redis 缓存、MySQL	缓存穿透、击穿、TTL、缓存一致性。
收藏资料	MySQL 唯一索引、Redis 热度	防重复收藏、唯一索引兜底、热度分数更新。
资料下载	Redis 限流、下载记录、定时统计	高频写库优化、Redis Hash 暂存、批量同步。
热门排行	Redis ZSet	score 设计、Top N 查询、下架资料过滤。

二、复习方法：从项目反推知识点

1. 先打开项目代码，从一个接口开始追踪：Controller -> DTO -> Service -> Mapper -> SQL / Redis。
2. 对每个技术栈整理 4 个问题：项目哪里用？为什么用？怎么实现？有什么风险和优化？
3. 看知识点串讲建立框架，再用八股题查漏补缺，最后把答案改成项目版。
4. 每个模块都准备 1 个“难点问题”和 1 个“优化方案”，不要只说实现了功能。

一个合格的项目化回答示例：

Redis 在我的项目里不是简单 set/get。资料详情用 String 缓存，热门资料和搜索词用 ZSet 做排行榜，下载限流用 String + TTL 计数器，下载次数用 Hash 暂存后定时批量同步到 MySQL。

三、Spring Boot 重点复习

项目使用场景：项目启动、自动配置、Controller/Service/Mapper 分层、统一返回、全局异常、参数校验、事务、拦截器。

1. 重点复习知识点

- Spring Boot 与 Spring、Spring MVC 的关系。
- @SpringBootApplication 的组成：@SpringBootConfiguration、@EnableAutoConfiguration、@ComponentScan。
- 自动配置的基本思路：根据依赖和配置创建 Bean。

- application.yml 配置读取，profiles 多环境配置。
- Bean 管理、依赖注入、组件扫描。
- 统一返回 Result、全局异常 @RestControllerAdvice。
- @Transactional 事务使用场景和失效场景。
- 拦截器 HandlerInterceptor 的执行时机。

2. 必会用法 / 关键 API

- @RestController、@RequestMapping、@GetMapping、@PostMapping。
- @Service、@Component、@Autowired / 构造器注入。
- @Configuration、@Bean。
- @Validated、@NotBlank、@NotNull。
- @Transactional 用于审核、收藏、下载记录等需要一致性的业务。

3. 高频面试追问

- Spring Boot 自动配置原理是什么？
- @SpringBootApplication 包含哪些注解？
- Controller、Service、Mapper 为什么要分层？
- @Transactional 什么情况下会失效？
- 拦截器和过滤器有什么区别？
- 全局异常处理怎么做？

4. 项目化回答方向

在项目中，Spring Boot 负责快速搭建后端应用，Spring MVC 负责接收 REST API 请求，Service 层处理资料上传、审核、下载等业务，Mapper 层访问 MySQL。登录接口通过拦截器校验 JWT，审核和下载记录使用事务保证数据一致性。

四、SSM / Spring MVC 重点复习

项目使用场景：HTTP 请求处理、参数绑定、JSON 返回、文件上传、拦截器认证、RESTful 接口设计。

1. 重点复习知识点

- Spring 负责 IOC / DI 和事务管理。
- Spring MVC 的请求流程：DispatcherServlet -> HandlerMapping -> HandlerAdapter -> Controller -> 返回结果。
- 参数绑定：@RequestBody、@RequestParam、@PathVariable。
- RESTful 接口风格：资源名用名词，动作通过 HTTP 方法表达。
- 文件上传 MultipartFile 的接收方式。
- 拦截器 preHandle / postHandle / afterCompletion。

2. 必会用法 / 关键 API

- POST /api/auth/login 接收 JSON 登录参数。

- POST /api/files/upload 使用 MultipartFile 接收文件。
- POST /api/admin/resources/{id}/audit 使用 PathVariable 接收资料 id。
- GET /api/resources/search 使用 RequestParam 接收关键词、分类、页码。

3. 高频面试追问

- Spring MVC 的请求处理流程？
- @RequestBody 和 @RequestParam 区别？
- RESTful 接口为什么比 /add、/delete 更规范？
- 如何实现文件上传？
- 拦截器如何获取当前用户？

4. 项目化回答方向

项目中的接口按资源划分，例如

/api/resources/search、/api/resources/{id}/download、/api/admin/resources/{id}/audit。这样比 add/delete/update/list 更清晰，也更符合后端工程规范。

五、MyBatis 重点复习

项目使用场景：用户、资料、文件、收藏、下载记录、审核记录等表的数据访问；资料搜索使用动态 SQL。

1. 重点复习知识点

- Mapper 接口与 XML 映射关系。
- MappedStatement 的概念：一个 SQL 对应一个映射语句。
- #{} 和 \${} 的区别：#{ } 预编译防注入，\${ } 字符串拼接有风险。
- resultMap 解决字段名和属性名不一致。
- 动态 SQL：if、where、trim、foreach、set。
- 分页查询与条件查询。
- 一级缓存、二级缓存了解即可。

2. 必会用法 / 关键 API

- ResourceMapper 根据 status、categoryId、keyword 查询资料。
- FavoriteMapper 插入收藏记录，配合唯一索引防重复。
- AuditRecordMapper 写入审核记录。
- DownloadRecordMapper 写入下载记录。

3. 高频面试追问

- #{} 和 \${} 有什么区别？
- MyBatis 如何防止 SQL 注入？
- Mapper 接口为什么不用写实现类？
- 动态 SQL 有哪些常用标签？
- resultMap 有什么作用？

- MyBatis 和 JPA 的区别？

4. 项目化回答方向

资料搜索接口中，MyBatis 动态 SQL 会根据关键词、分类、课程名等条件拼接查询，同时固定加 `status = 1`，保证用户只能搜索审核通过的资料。对于用户输入的关键词使用 `#{}` ，避免 SQL 注入。

六、MySQL 重点复习

项目使用场景：存储用户、资料、文件、审核、收藏、下载记录；用索引优化查询；用事务保证状态变更一致性。

1. 重点复习知识点

- 表设计：主键、逻辑外键、状态字段、创建时间、更新时间。
- 索引类型：主键索引、唯一索引、普通索引、联合索引。
- 联合索引最左前缀原则。
- 索引失效场景：函数、隐式类型转换、左模糊 like、范围条件后列不一定生效。
- 事务 ACID：原子性、一致性、隔离性、持久性。
- 隔离级别：读未提交、读已提交、可重复读、串行化。
- MVCC、undo log、redo log、binlog 的基本作用。
- explain 分析查询计划。

2. 必会用法 / 关键 API

- favorite 表 `user_id + resource_id` 唯一索引，防止重复收藏。
- file_info 表 `file_md5` 索引，用于文件去重。
- resource 表 `status + category_id` 联合索引，用于查询审核通过的分类资料。
- 审核操作使用事务：更新 resource 状态并写入 audit_record。

3. 高频面试追问

- 为什么要给 favorite 表加唯一索引？
- 联合索引最左前缀原则是什么？
- MySQL 默认隔离级别是什么？
- MVCC 解决了什么问题？
- explain 主要看哪些字段？
- 事务失效或回滚失败可能是什么原因？

4. 项目化回答方向

我给 favorite 表设计 `user_id + resource_id` 唯一索引，即使用户重复点击收藏或并发请求，数据库也能兜底防止重复收藏。file_info 表的 `file_md5` 索引用于上传时快速判断文件是否已存在，避免重复存储。

七、Redis 重点复习

项目使用场景：资料详情缓存、热门资料排行榜、热门搜索词、下载限流、下载量临时统计、登录态缓存。

1. 重点复习知识点

- Redis 为什么快：内存存储、单线程避免锁竞争、IO 多路复用、高效数据结构。
- String、Hash、List、Set、ZSet 的典型使用场景。
- ZSet：score 排序，适合排行榜。
- Hash：适合存对象字段或聚合统计。
- TTL：缓存过期、限流窗口。
- 缓存穿透、击穿、雪崩及解决方案。
- Redis 和 MySQL 一致性：先更新数据库，再删除缓存；设置 TTL 兜底。
- BigKey、热 Key、缓存预热、缓存降级。
- RDB 和 AOF 持久化区别。

2. 必会用法 / 关键 API

- resource:detail:{id} 缓存资料详情。
- rank:resource:hot 使用 ZSet 存热门资料。
- rank:search:keyword 使用 ZSet 存热门搜索词。
- limit:download:{userId} 使用 String + TTL 做 1 分钟下载次数限制。
- stat:download:resource 使用 Hash 暂存资料下载量。

3. 高频面试追问

- Redis 为什么快？
- ZSet 如何实现排行榜？
- 缓存穿透、击穿、雪崩怎么解决？
- Redis 和 MySQL 如何保证一致性？
- 为什么下载量不直接写 MySQL？
- Redis 宕机怎么办？
- BigKey 和热 Key 怎么处理？

4. 项目化回答方向

项目中 Redis 主要承担读写压力削峰。资料详情用缓存减少 MySQL 查询；热门资料和热门搜索词用 ZSet 做 Top N；下载接口用 String + TTL 限流；下载量用 Hash 暂存，再由定时任务批量同步到 MySQL，降低高频写库压力。

八、JWT / 登录认证 / 权限控制重点复习

项目使用场景：登录成功后生成 Token，请求携带 Token，拦截器解析用户身份，管理员接口做角色校验。

1. 重点复习知识点

- JWT 三部分：Header、Payload、Signature。
- Payload 不能存敏感信息，因为只是编码不是加密。
- Token 过期时间、刷新机制、退出登录处理。
- JWT 与 Session 的区别。
- 拦截器解析 Token 后如何保存当前用户，例如 ThreadLocal。
- RBAC 基础：用户、角色、权限。
- 密码不能明文存储，应使用 BCrypt 等哈希方式。

2. 必会用法 / 关键 API

- POST /api/auth/login 校验密码后生成 JWT。
- Authorization: Bearer token 请求头传递 Token。
- LoginInterceptor 解析 Token 并保存 userId、role。
- 管理员审核、下架接口检查 role 是否为 admin。

3. 高频面试追问

- JWT 由哪几部分组成？
- 为什么 JWT 的 Payload 不能放密码？
- JWT 和 Session 有什么区别？
- Token 被盗怎么办？
- 如何实现管理员权限控制？
- ThreadLocal 使用后为什么要 remove？

4. 项目化回答方向

项目中普通学生只能上传、搜索、下载和收藏资料，管理员才能审核和下架资料。请求进入后由拦截器解析 JWT，获取用户 ID 和角色，再对管理员接口做权限校验。

九、文件上传 / MD5 去重重点复习

项目使用场景：学生上传 pdf、doc、ppt、zip 等学习资料；系统校验文件、计算 MD5、防止重复保存。

1. 重点复习知识点

- MultipartFile 接收文件。
- 文件大小限制和后端校验。
- 文件类型校验：扩展名和 MIME 类型都不能完全信任。
- 文件名安全：重命名存储，避免路径穿越和重名覆盖。
- MD5 摘要计算，用于判断文件内容是否重复。
- 文件元数据单独建 file_info 表。
- 文件存储位置与数据库记录的关系。
- 下载时不应直接暴露服务器真实路径。

2. 必会用法 / 关键 API

- 上传前校验后缀、大小。
- 计算文件 MD5，查询 file_info 是否已存在。
- 已存在则复用 fileId，不重复保存物理文件。
- 不存在则保存到 uploads 目录并写入 file_info。
- resource 表引用 file_id，并进入待审核状态。

3. 高频面试追问

- 如何实现文件上传？
- 为什么要计算 MD5？
- MD5 去重有什么局限？
- 为什么文件表和资料表分开？
- 如何防止恶意文件上传？
- 下载接口如何保证安全？

4. 项目化回答方向

我将文件信息和资料信息拆成 file_info 与 resource 两张表。用户上传时先校验文件，再计算 MD5。如果已有相同 MD5 文件，则不重复保存物理文件，只创建新的资料记录，减少存储浪费。

十、定时任务 / 异步处理重点复习

项目使用场景：将 Redis 中临时累计的下载量定时批量同步到 MySQL；后续可扩展 MQ 异步同步搜索索引。

1. 重点复习知识点

- @EnableScheduling 开启定时任务。
- @Scheduled fixedRate、fixedDelay、cron 的区别。
- 定时同步为什么能降低数据库写压力。
- 批量更新 MySQL 的注意点。
- 任务失败重试、幂等设计。
- 多实例部署下定时任务重复执行问题。
- MQ 可选场景：下载统计、索引同步、审核通知。
- 消息可靠性基本概念：确认、重试、死信、幂等消费。

2. 必会用法 / 关键 API

- 每隔一段时间读取 stat:download:resource。
- 批量更新 resource.download_count。
- 同步成功后清理或扣减 Redis 中对应统计值。
- 审核通过后可异步同步搜索索引。

3. 高频面试追问

- @Scheduled 怎么使用?
- cron 表达式怎么写?
- 为什么下载量先写 Redis 再同步 MySQL?
- 定时任务失败怎么办?
- 多台机器同时执行定时任务怎么办?
- 为什么引入 MQ?

4. 项目化回答方向

下载接口属于高频操作，如果每次下载都 update MySQL，会造成写压力。我把下载次数先累加到 Redis Hash，由定时任务批量同步到 MySQL，牺牲一点实时性换取更好的吞吐。

十一、Maven / Git / 工程化重点复习

项目使用场景：项目依赖管理、启动构建、版本控制、提交记录、协作开发习惯。

1. 重点复习知识点

- pom.xml 的作用：项目坐标、依赖、插件、构建配置。
- Maven 生命周期：clean、compile、test、package、install。
- dependency scope：compile、test、provided、runtime。
- 依赖冲突与依赖树。
- .gitignore 忽略 target、logs、uploads、IDE 配置等。
- Git 基本命令：add、commit、branch、merge、status、log。
- 规范提交信息：docs、feat、fix、refactor、test。

2. 必会用法 / 关键 API

- 用 Maven 管理 Spring Boot、MyBatis、MySQL、Redis、JWT 等依赖。
- 每完成一个阶段提交一次 Git：需求文档、表设计、登录、上传、审核、Redis 优化。
- README 中写清启动步骤、数据库初始化、Redis 配置。

3. 高频面试追问

- Maven 的 dependency 和 plugin 有什么区别?
- Maven 生命周期有哪些?
- 如何解决依赖冲突?
- Git 如何处理冲突?
- 为什么要写 .gitignore?
- 项目提交记录如何体现你独立开发?

4. 项目化回答方向

我把项目拆成多个阶段提交，例如 docs: add api design、feat: implement resource upload、feat: add redis hot ranking。这样不只是最终有代码，也能体现完整的开发过程。

十二、项目综合难点复习

难点	问题本质	你的项目回答方向
如何避免普通 CRUD	不能只说增删改查，要突出业务流程和技术设计。	强调上传审核状态流转、文件 MD5 去重、Redis 排行榜、下载限流、下载量异步统计。
Redis 与 MySQL 一致性	缓存和数据库可能出现短暂不一致。	资料详情更新时先更新数据库，再删除缓存；设置 TTL 兜底；重要数据以 MySQL 为准。
下载量高频写库	每次下载都 update 会增加 MySQL 写压力。	先写 Redis Hash，定时批量同步 MySQL，牺牲实时性换吞吐。
重复收藏	前端防抖不可靠，并发请求仍可能重复。	业务层先查，数据库层用 user_id + resource_id 唯一索引兜底。
文件重复上传	相同文件多次上传浪费存储。	计算 MD5，file_info 表索引查询，已存在则复用 fileId。
非法审核状态	不能从任意状态随便改成任意状态。	设计状态流转规则，待审核才能通过/拒绝，通过后才能下架，事务记录审核日志。

十三、面试前 30 分钟速记清单

- 一句话介绍项目：面向学生的资料上传、审核、搜索、下载、收藏和热门排行系统。
- 核心技术栈：Spring Boot + MyBatis + MySQL + Redis + JWT。
- 核心业务流程：上传文件 -> MD5 去重 -> 创建资料 -> 管理员审核 -> 用户搜索/下载/收藏。
- 核心 Redis 场景：资料详情缓存、热门资料 ZSet、热门搜索词 ZSet、下载限流、下载量 Hash 统计。
- 核心 MySQL 设计：file_info 存文件元信息，resource 存资料信息，favorite 唯一索引防重复，audit_record 记录审核日志。
- 最大难点准备：下载量为什么不直接写 MySQL，Redis 和 MySQL 如何保持一致，如何防止重复收藏和非法审核状态。
- 一句话说明非 CRUD：项目包含文件去重、审核状态流转、排行榜、限流、异步统计等真实业务和性能设计。

十四、7 天复习安排

时间	复习主题	输出物
第 1 天	项目整体、Spring Boot、SSM 请求流程	项目架构图 + 请求链路说明。
第 2 天	MyBatis、MySQL 表设计、索引、事务	数据库表说明 + 索引设计说明。
第 3 天	Redis 数据结构、缓存、排行榜、限流	Redis key 设计表 + 20 个 Redis 问题。
第 4 天	JWT、拦截器、权限控制	登录认证流程图 + 管理员权限说明。

第 5 天	文件上传、MD5 去重、审核状态流转	上传审核流程图 + 状态流转表。
第 6 天	定时任务、下载统计、项目优化点	项目难点 5 个：问题 -> 方案 -> 缺点 -> 优化。
第 7 天	模拟面试和简历表达	项目面试问答文档 + 简历 4 条 bullet。

十五、可让 Codex 继续辅助生成的材料

你可以把以下提示词交给 Codex，让它结合你的实际代码生成更贴近项目的复习资料。

- 请阅读当前项目代码，站在 Java 后端实习面试官角度，围绕 Spring Boot、MyBatis、MySQL、Redis、JWT、文件上传、限流、排行榜提出 30 个追问，并结合具体类、方法、表、Redis key 给出参考回答。
- 请根据当前项目生成 docs/interview-cheatsheet.md。每个技术栈包括：项目使用位置、核心原理、关键代码位置、面试追问、项目版回答。内容要适合面试前 30 分钟快速复习。